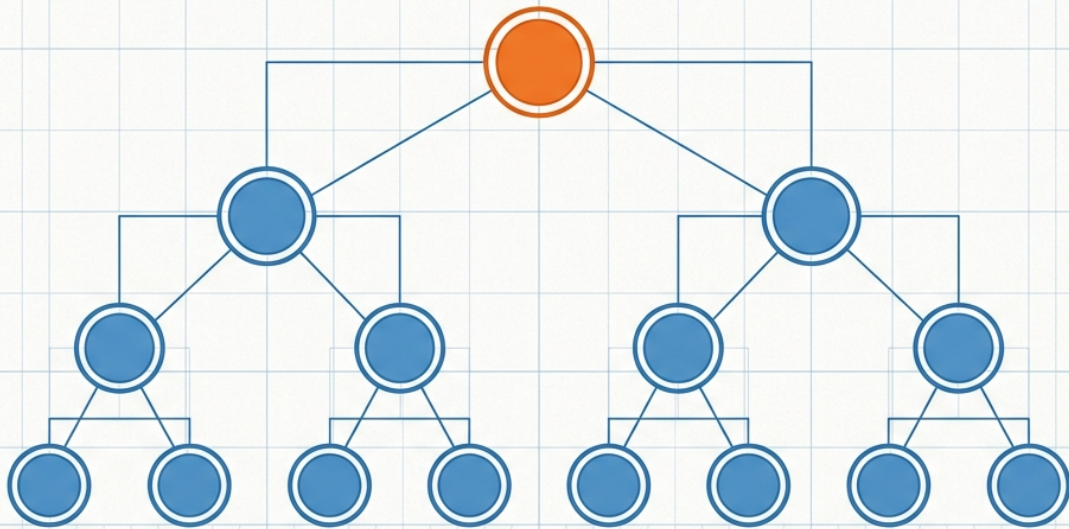


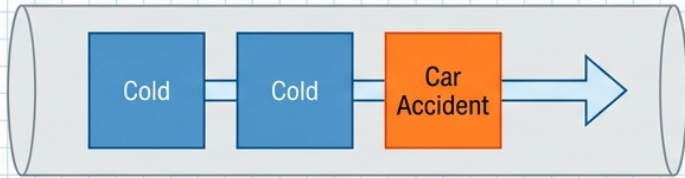


Priority Queues & Heaps

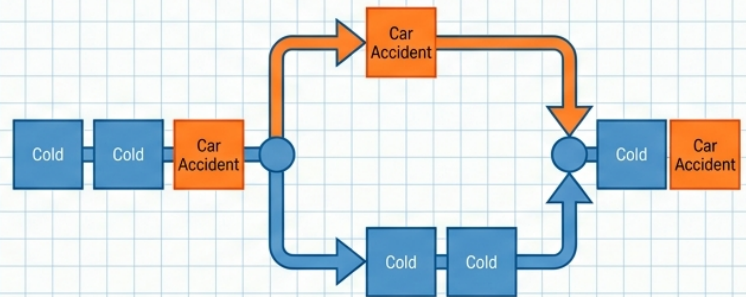
The Algorithmic Blueprint for Coding Interviews

The Problem with Standard Queues

FIFO Line



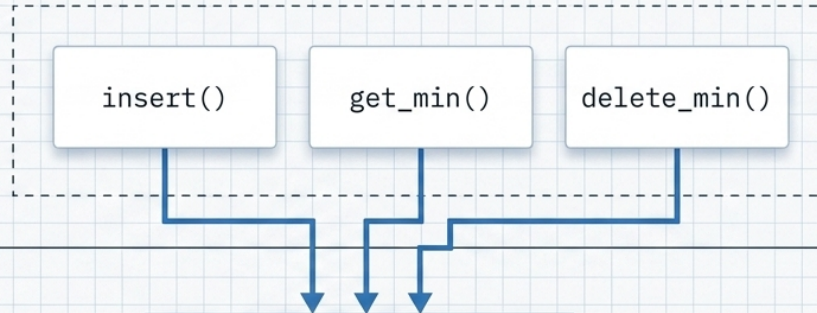
Priority Queue



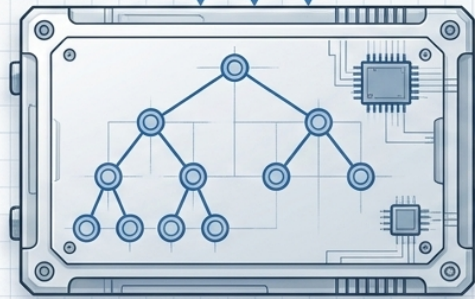
A standard queue operates on a **First-In-First-Out (FIFO)** basis. But real-world systems require sorting by urgency, not just arrival time. In a hospital triage, a patient with a severe injury instantly bypasses a patient with a cold, regardless of who arrived first. **Severity is the key.**

Abstract Idea vs. Concrete Engine

Priority Queue
(Abstract Data Type)



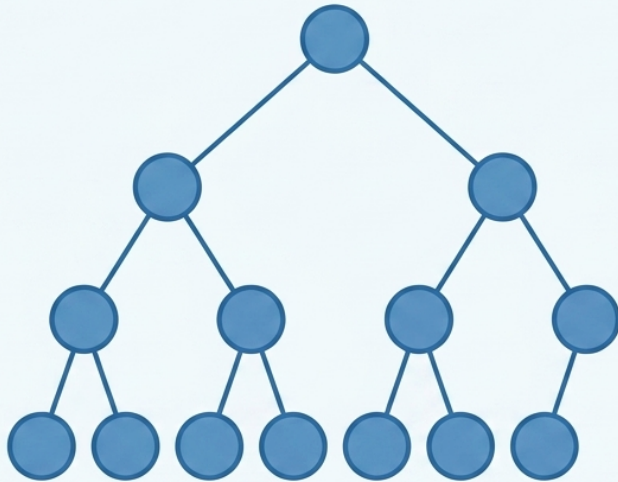
The Heap
(Concrete Data Structure)



A Priority Queue is simply an Abstract Data Type—a set of rules and expected behaviors.

The Heap is the concrete data structure we use to actually implement those rules efficiently under the hood.

The Two Non-Negotiable Rules

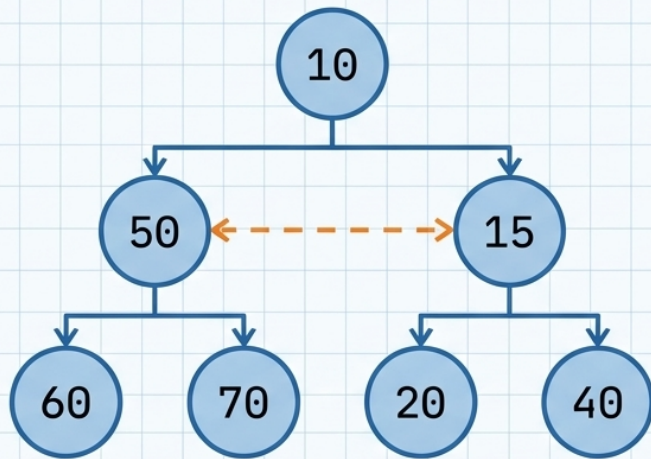


A valid binary heap must strictly obey two properties at all times:

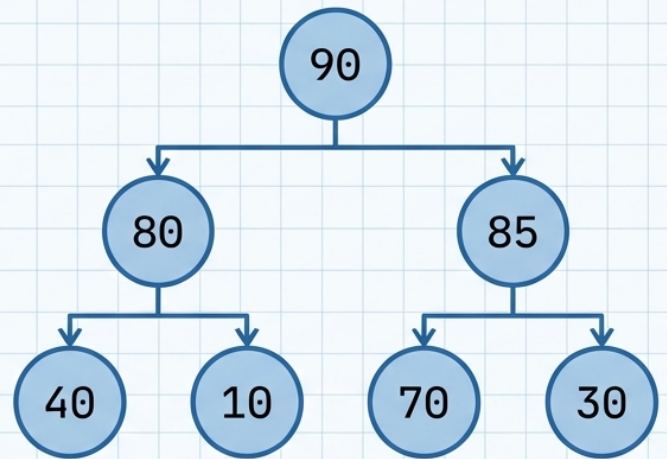
- ✓ 1. The **Shape Rule**: It must be a **complete tree**. Every level is fully populated except possibly the last, which is filled strictly left-to-right.
- ✓ 2. The **Order Rule**: For a **Min-Heap**, every parent node's key is strictly less than or equal to its children's keys. (For a **Max-Heap**, it is greater than or equal to).

Min-Heap vs. Max-Heap

Min-Heap: Parents $\leq$ Children

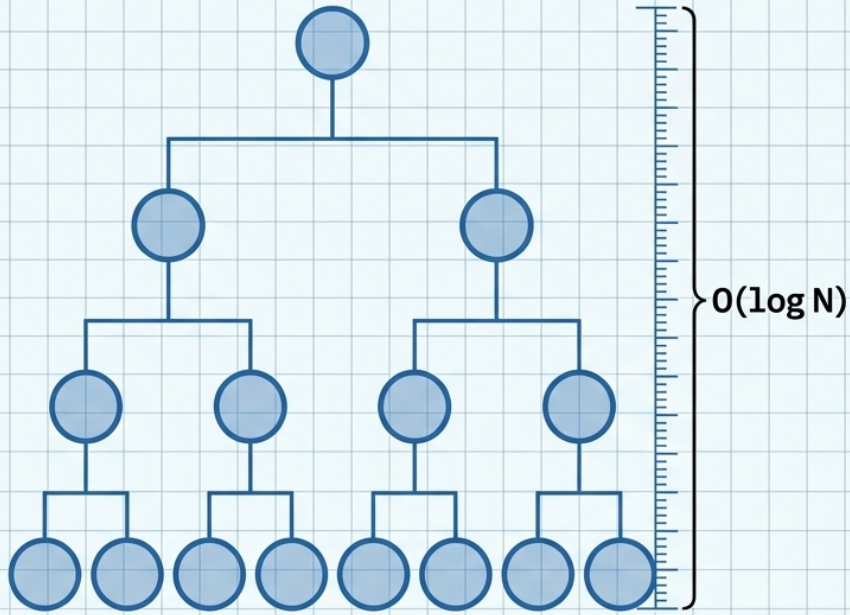


Max-Heap: Parents $\geq$ Children



Crucial Insight: Unlike a Binary Search Tree, a heap is NOT sorted horizontally across a level. The only guaranteed relationship is vertical (parent-to-child).

The Efficiency of the Engine



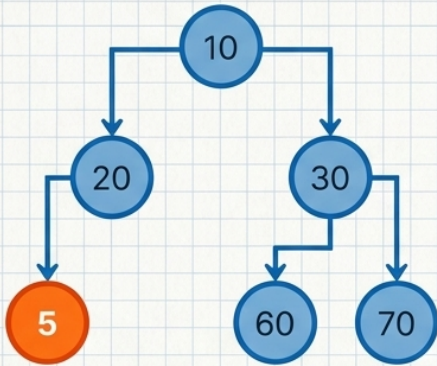
What makes heaps so useful?

- Instant $O(1)$ access to the minimum (or maximum) value at the root.
- When inserting or deleting, the structure must repair itself.
- Because the tree strictly adheres to the Complete Shape Rule, its height is firmly locked at $\log(N)$.
- Therefore, repair operations—traveling up or down the tree—never take longer than $O(\log N)$.

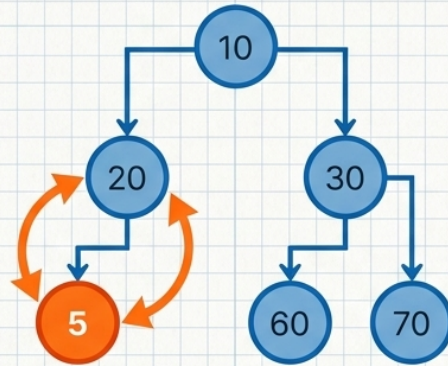
Insertion Mechanics: Bubble Up

To insert a key, place it at the next available slot at the **bottom level** to maintain the complete tree shape. Then, compare it with its parent and swap upward until the order rule is restored.

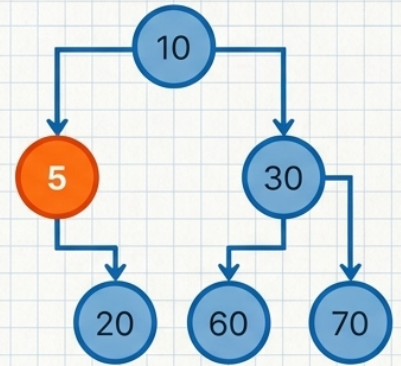
Step 1: Insert at Bottom



Step 2: Compare & Swap

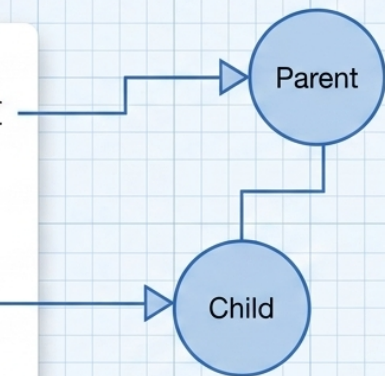


Step 3: Restored Order



The Bubble Up Implementation

```
public void bubbleUp(Node node) {  
    while (node.parent != null && node.parent.key > node.key) {  
        // swap node and node.parent  
        int temp = node.key;  
        node.key = node.parent.key;  
        node.parent.key = temp;  
        node = node.parent;  
    }  
}
```

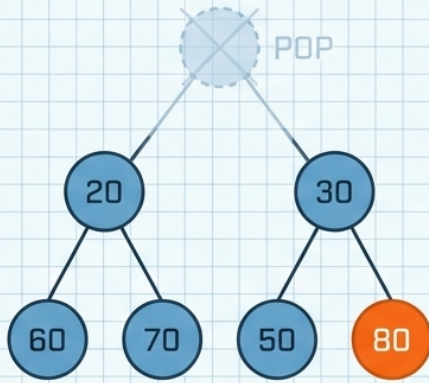


The algorithm iteratively checks the parent and swaps, shifting the pointer upwards until the root is reached or the parent is smaller.

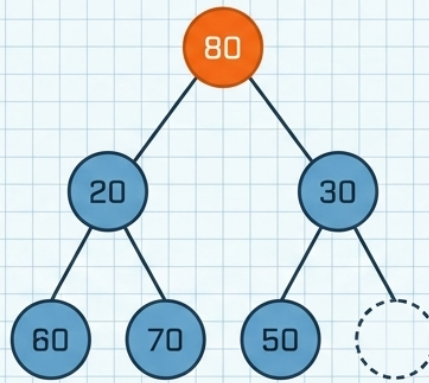
Deletion Mechanics: Bubble Down

To delete the minimum value, we pop the root. To repair the tree, we take the last element in the heap and place it at the root. Then, we “bubble down” by continually swapping it with its SMALLEST child until the order is correct.

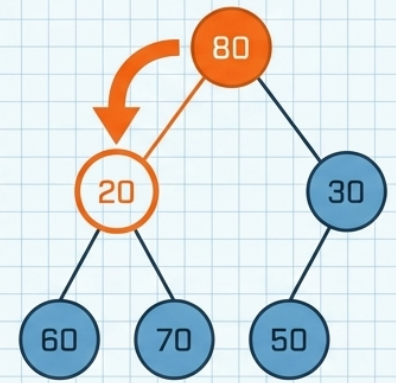
Step 1: Pop & Identify Last



Step 2: Move to Root



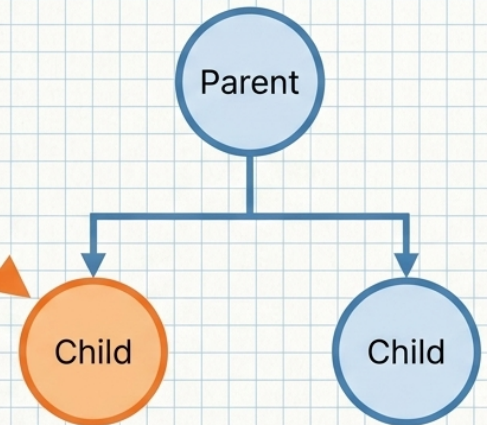
Step 3: Swap with Smallest Child



Swap with smallest child (20)

The Bubble Down Implementation

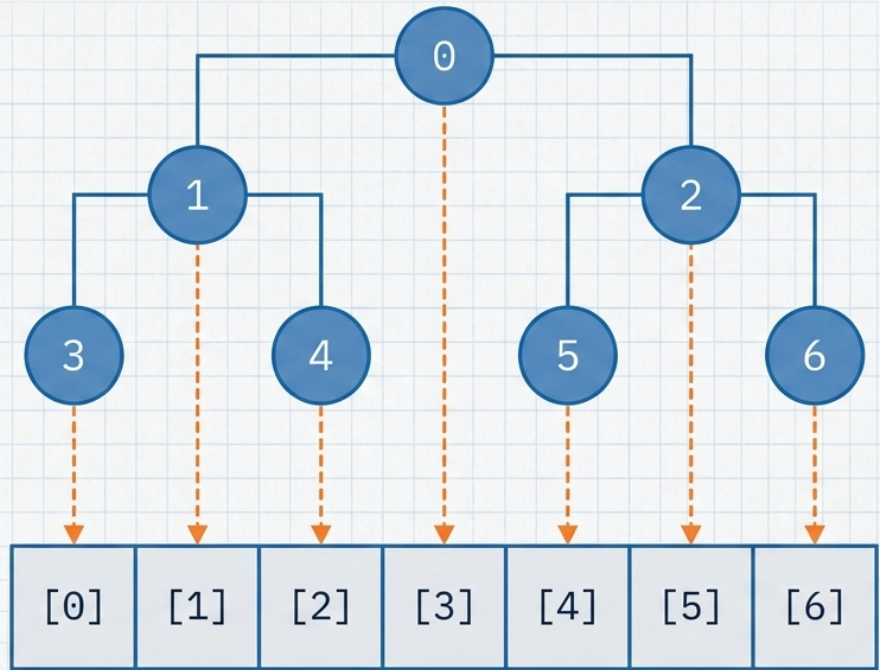
```
public void bubbleDown(Node node) {  
    while (!node.isLeaf()) {  
        Node smallestChild = node.getSmallestChild();  
        if (smallestChild.key < node.key) {  
            int temp = node.key;  
            node.key = smallestChild.key;  
            smallestChild.key = temp;  
            node = smallestChild;  
        } else { break; }  
    }  
}
```



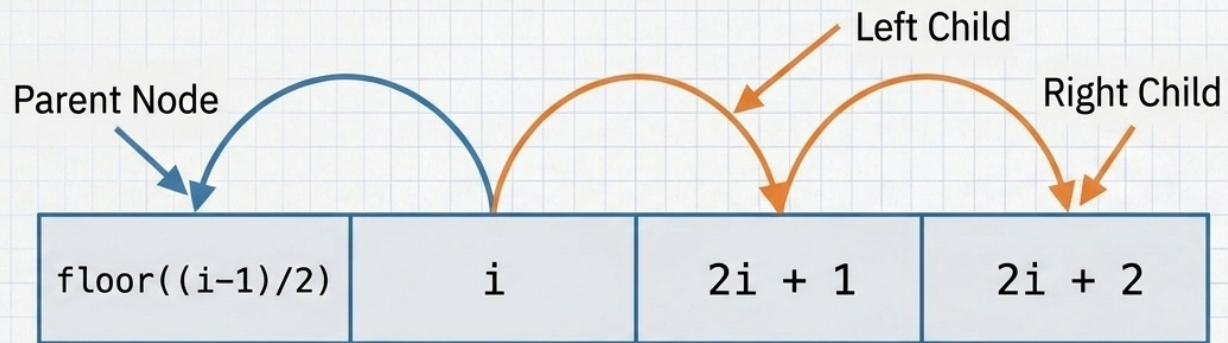
The Array Secret

The biggest trick in interviews: Heaps are drawn as trees on whiteboards, but they are coded as flat arrays.

Because the tree is perfectly complete, we don't need objects or pointers. We store data compactly and navigate using pure index arithmetic.



Navigating the Array Math



Pointers are completely replaced by index math. For any node sitting at index i :

- Left Child: exactly at index $2i + 1$
- Right Child: exactly at index $2i + 2$
- Parent Node: exactly at index $\text{floor}((i-1)/2)$

(Note: For a k -ary heap, children are at $ki+1$ to $ki+k$, and the parent is at $\text{floor}((i-1)/k)$)

The Interview Survival Matrix

Python

```
import heapq
```

Min-Heap default.

Java

```
java.util.PriorityQueue
```

Min-Heap default.

C++

```
std::priority_queue
```

MAX-HEAP DEFAULT! 

JavaScript

No native support. Must build a custom class or use external libraries.

Python Tactics: heapq

Snippet 1: Insertion

```
heapq.heappush(h, (5, 'write code'))
```

Snippet 2: Transformation

```
heapq.heapify(arr)
```

Python's built-in `heapq` module is a Min-Heap by default.

Trick 1 (Tuples):

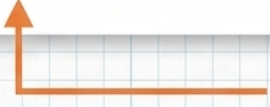
Push tuples to sort by complex criteria. Python automatically compares position by position, starting with the first element.

Trick 2 (heapify):

If you have all elements upfront, `heapq.heapify(arr)` converts a list to a heap in strict **$O(N)$** time. This is mathematically significantly faster than manually pushing elements one by one (**$O(N \log N)$**).

Java Tactics: PriorityQueue & Comparators

```
PriorityQueue<Task> tasks = new PriorityQueue<>(5, Comparator.comparingInt(t -> t.priority));
```



Java's PriorityQueue defaults to a Min-Heap using natural ordering.

When dealing with custom objects that don't inherently implement the Comparable interface, you must inject a custom Comparator via a lambda expression at construction time to dictate the strict sorting logic.

The Inversion Protocol

Python / JS Workaround	Java	C++
The -1 Hack. Reverse the sign of numbers before pushing: <code>heappush(h, -1 * val)</code>. Reverse again when popping. This turns a Min-Heap into a Max-Heap.	<code>Collections.reverseOrder()</code> Pass this directly into the <code>PriorityQueue</code> constructor to invert the natural Min-Heap the natural Min-Heap behavior to a Max-Heap.	<code>std::greater<T></code> C++ defaults to a Max-Heap (using <code>std::less</code>). To create a Min-Heap, you must explicitly pass <code>std::greater<int></code> as the third template argument.

Application: Top 'K' Elements

Prompt: Given an unsorted list of numbers, return the top 3 smallest numbers.

Sorting

Sort the entire array, slice the first 3.

Time Complexity: $O(N \log N)$



Priority Queue (Heap)

Throw elements into a Min-Heap, pop 3 times.

Time Complexity: $O(N \log K)$



The Priority Queue is the definitive solution for any 'Merge K', 'Top K', or 'Smallest K' problem, dramatically reducing time complexity when K is much smaller than N.

The Elegant Execution

```
public static List<Integer> heapTop3(List<Integer> arr) {  
    PriorityQueue<Integer> heap = new PriorityQueue<>();  
    for (int x : arr) { heap.add(x); }  
    List<Integer> res = new ArrayList<>();  
    for (int i = 0; i < 3; i++) { res.add(heap.poll()); }  
    return res;  
}
```

$O(\log N)$ insertion operation inside a loop.

"Bubble Down" abstracts away the structural repair, guaranteeing we always pull the absolute minimum in $O(\log N)$ time without needing to manually resort the array.

The Algorithmic Blueprint: Priority Queue

The Concept

An Abstract Data Type **(Priority)** backed by a Complete Binary Tree (Heap).

Rules: Complete Shape + Parent Order.

The Math

Array Implemented.

Left Child = $2i + 1$

Right Child = $2i + 2$

Parent = $\text{floor}((i-1)/2)$

The Complexity

- Find Min/Max: $O(1)$
- Insert (Bubble Up): $O(\log N)$
- Delete (Bubble Down): $O(\log N)$
- Heapify (Array to Heap): $O(N)$